AtomTracker

In-House Goal Setting & Tracking Portal — Hackathon Submission

Generated 2026-05-18 · Single-document submission (live deployment, source repository, architecture).

1. Live working deployment

Production frontend (Vercel), API (Render), and interactive OpenAPI docs. The Render free tier may cold-start after idle periods; the first request can take about 30 seconds — refresh once if needed.

Component	URL
Frontend (SPA)	https://atom-tracker-rust.vercel.app
Backend API	https://atomtracker.onrender.com
API documentation	https://atomtracker.onrender.com/docs
Demo seed (idempotent)	https://atomtracker.onrender.com/setup-demo

Clickable links (same URLs as above):

https://atom-tracker-rust.vercel.app

https://atomtracker.onrender.com

https://atomtracker.onrender.com/docs

https://atomtracker.onrender.com/setup-demo

Demo login credentials

Role	Email	Password
Admin	admin@test.com	admin
Manager	manager@test.com	manager
Employee	employee@test.com	employee

2. Source code repository

Full-stack implementation (FastAPI + SQLite backend, React + Vite frontend). README contains local setup, demo flow, and feature overview.

https://github.com/dharmendra26-wiz/AtomTracker

Repository layout

```
atomtracker/
  backend/    # FastAPI, SQLAlchemy, JWT
  frontend/  # React 19, Tailwind v4, react-router
```

3. System architecture

Two-tier architecture: React SPA communicates with FastAPI over REST/JSON. Authentication uses HS256 JWTs in the **Authorization: Bearer** header; RBAC is enforced server-side. Persistence is SQLite via SQLAlchemy (Postgres-ready via configuration). Shared KPIs use a **source_goal_id** self-reference on goals so cascaded copies inherit check-ins from the primary goal.

Figure A — High-level system architecture

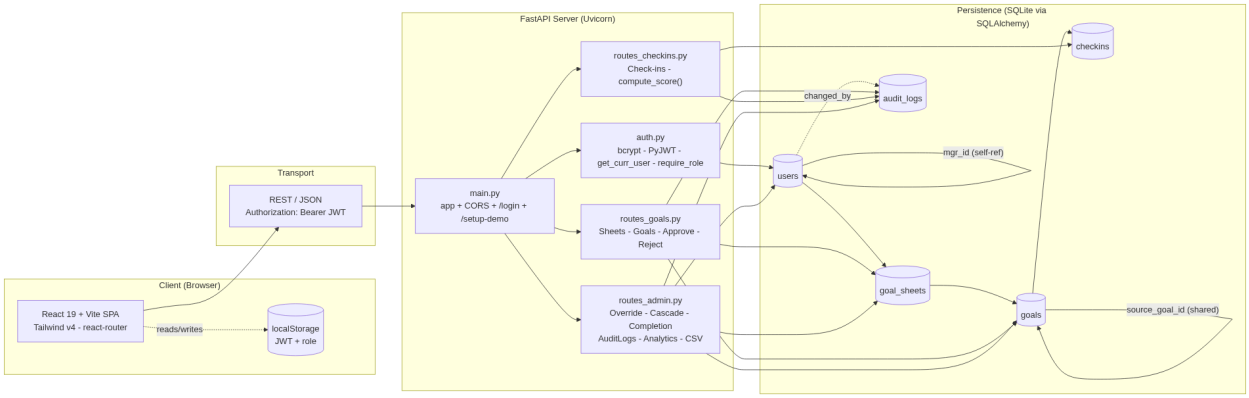
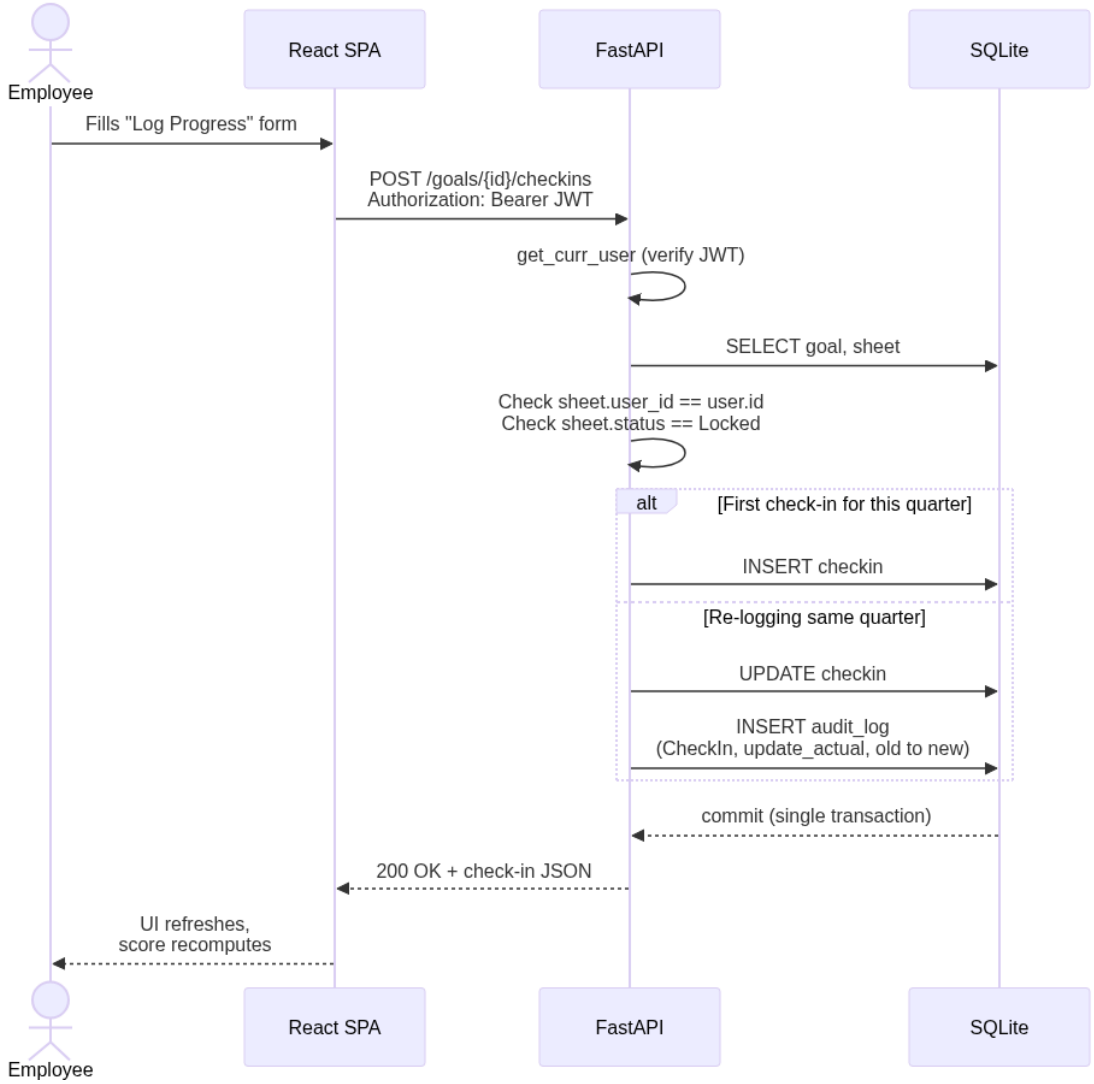


Figure B — Request lifecycle (example: employee logs quarterly progress)



Architectural decisions (summary)

Decision	Rationale
JWT in Authorization header	Stateless API; simple CORS; no cookie CSRF surface
Generic AuditLog (entity_type, entity_id)	One pattern for sheets, goals, check-ins, cascades; easy to extend
Audit in same DB transaction	Change and log commit together — no orphan audit rows
Shared goals via source_goal_id	Single source of truth for actuals; copies stay aligned automatically
Server-side validation	UI hints only; weights, roles, and ownership re-checked on every request

Appendix — Requirements traceability

Mapping from the hackathon problem statement to shipped capabilities (representative API surface).

Requirement	Shipped	API / mechanism
RBAC: Employee, Manager, Admin	Yes	JWT + require_role
Up to 8 goals; min weight 10; total 100%	Yes	POST /sheets/{id}/goals, .../submit
Manager approves / locks sheet	Yes	POST /sheets/{id}/approve
Return for rework with comment	Yes	POST /sheets/{id}/reject
Manager pre-approval edits (target/weight)	Yes	POST /goals/{id}/override
Quarterly check-ins + scoring by UoM	Yes	POST /goals/{id}/checkins, GET .../progress
Manager feedback on check-in	Yes	POST /checkins/{id}/comment
Shared / cascaded KPIs	Yes	POST /goals/{id}/cascade + source_goal_id
Admin override + audit trail	Yes	override + AuditLog rows
Completion visibility by quarter	Yes	GET /completion
Audit explorer + CSV export	Yes	GET /audit-logs*, GET /reports/achievements.csv
Org analytics	Yes	GET /analytics

Built with: FastAPI · SQLAlchemy 2 · PyJWT · bcrypt · React 19 · Vite 8 · Tailwind CSS v4 · react-router-dom v7